

Big Data Assessed Exercise (2025) - Feedback

Siddhartha Pratim Dutta (2897074d)

2897074d@student.gla.ac.uk

Moodle Participant ID: 112521441

1 Introduction

For this exercise you needed to implement a Java-based Spark solution to produce a non-personalised financial asset recommender. We were testing your ability to produce a high-quality solution, this means that it should produce the correct answer (there were multiple of these depending on some design decisions I will cover later) and be efficient in terms of time and resources consumed.

Marking was performed in stages. First, submissions were allocated to the marking team (the course coordinator (Richard) and a set of marking assistants). The assigned marker downloaded your solution, loaded it into an integrated development environment, compiled it, ran it, and analyzed your code. They assigned the first 15 of the 25 marks. Assuming you submitted something, you will likely have received some marks from this portion, as receiving marks here are based on whether the submitted code included the expected components of a workable solution, but do not require a complete solution. Marks were also assigned here for code documentation and perceived scalability. The marker recorded marks against a declarative marking scheme which you can find later in this document, along with their run-time statistics (referred to as 'manual run-time').

After manual marking was completed, all solutions were passed to the course coordinator to execute the correctness and efficiency evaluation. Richard created an automatic script that decompressed your solution, copied the source directory into a fresh copy of the project template, compiled (more precisely packaged into a runnable Jar) your solution, and then executed it on a standard machine. Run-time, memory consumption and data input and shuffle statistics were recorded. The output of the program was then automatically compared against the series of valid solutions, where marks were assigned for the correct ranking of assets, as well as correct technical indicator and asset metadata. Finally, all submitted solutions were compared on run-time, memory and data statistics, solutions were grouped into efficiency categories and assigned marks. If a solution failed to extract or compile, it was manually checked by the course coordinator. If the solution could be made to work with only minor changes (e.g. adding a missing semi-colon, or fixing imports), then those changes were made and the solution re-executed (this is why returning the marks was a couple of days late).

2 Compile Failures

During marking we have made a best effort to get a partially broken solutions to produce some output. However, if you find that in this document your solution did not compile in our tests but it did for you before submission, then send an email to:

- richard.mccreadie@glasgow.ac.uk

and summarize your execution configuration (particularly note the Java and Spark version you used).

3 Solution Components (10 Marks)

Component	What to Look For	Mark Breakdown	Available	Mark Awarded
Time Filtering	Solutions should filter the input pricing data file to only contain data from the year before the given target date.	1 Mark for filtering out data prior to 2019-04-01. 1 Mark for filtering out data after 2020-04-01. For both of these we do not care about differences from date inclusivity vs. exclusivity.	2	2
Asset Filter	The asset metadata file contains assets that cannot ever reach the final ranking, and so these should be filtered out, namely assets missing either a name or p/e ratio.	1 Mark for removing assets missing data. Filtering should be done early.	1	1
Group Prices	Students will need to group the price data by asset (ticker). This can be done via groupByKey or aggregateByKey.	1 Mark for grouping by asset	1	1
Technical Indicator Calculation	Students need to apply the provided Volatility and Returns calculators for each of the assets.	1 Mark for calculating both metrics	1	1
Volatility Filter	Once volatility has been calculated, this needs to be used to filter out overly volatile assets (volatility>4)	1 Mark for filtering the dataset	1	1
Broadcast Asset Metadata	The final filtering step involves removing assets with very high price to earnings ratios (>25). To do this, the student needs to provide access to the p/e ratio data in a price dataset transformation, this should be broadcast to avoid data shuffles.	1 Mark for correctly broadcasting the needed asset metadata.	1	1
Filter by P/E Ratio	For each asset, now that the asset metadata is available, the student should filter the assets using the p/e ratio, and also store the names, as they will need that in the final return.	1 Mark for filtering the by p/e ratio.	1	1
Rank by Returns	The final preparation step is to rank by the Returns for each asset, the best way to do this in spark is via a reducer.	1 Mark for doing this in spark via reducer or other process that does not collect data at the driver.	1	1
Asset Return	Students need to return data in the specified format, including all the fields	1 Mark for having 5 assets returned that include the ticker, name and features.	1	1
Total			10	10

Table 1: Solution Components Marking Scheme and Marks

4 Code Quality and Scalability (5 Marks)

Component	What to Look For	Mark Breakdown	Available	Mark
In-line Comments	The student should provide in-line comments for the main rankInvestments method, see Richard's solution for an example of detail.	1 Mark for comments, a half mark can be awarded for partial comments or poor detail.	1	1
Class Comments	The students will need to implement a range of transformations, which will typically be represented via separate classes that are called.	1 Mark for class documentation, i.e. what each transformation class does.	1	1
Scalability	There are various possible solutions for the Big Data exercise that will produce the correct answer, however some will scale-up better than others. There are three extra marks for having a solution that will scale efficiently.	Start with three marks and then remove one mark for each issue you identify in the students code that might make it scale poorly, examples include: - Failing to filter the price data early - Using dataset joins rather than broadcast variables - Keeping unneeded data longer than needed - Collecting too much data at the driver program, or creating large objects in a single transformation	3	3
Total			5	5

Table 2: Code Documentation and Scalability Marking Scheme and Marks

5 Run-time Performance

Table 3 reports run-time statistics regarding the execution time and resources used by your solution. I have included the benchmark column, which is the data for the course-coordinator's solution, it is possible to produce a faster solution than this. The manual column is the data reported by the manual assessor, the automatic column is the data reported by the automatic test script. The metrics are as follows:

- **End-to-End Time:** This is the time taken by the solution from the creation of the spark session to returning the ranked list (does not include any Thread.sleep()).
- **Task Time:** This is the total reported computation time by the executor (worker) that ran the different tasks in your solution. It is the sum of task time across all (4) threads in the executor, hence why it is larger than the end-to-end time.

- **Garbage Collection (GC) Time:** This is the total reported time spent by the executor (worker) across threads cleaning up unneeded java objects.
- **Input Read:** This is the amount of data that Spark needed to read in (from disk) during the computation. The benchmark reads each file only once, if your solution reports more than this, then data needed to be re-read.
- **Data Shuffled:** This is the amount of data that had to be sorted and moved from one task to another (in this case only crossing threads, but in a distributed deployment this often would result in network transfer).

If performance is listed as 'DNC', this indicates that the submitted code either did not compile or complete within 3 minutes.

Metric	Source	Benchmark	Manual	Automatic
End-to-End Time	SPARK.DONE	29s	-	28s
Task Time	SPARK.DATA	33s	41s(0.7s)	33s
GC Time	SPARK.DATA	0s	-	(0s)
Input Read	SPARK.DATA	2382MB	2.3GB	2391MB
Data Shuffled	SPARK.DATA	7MB	17.1MB	17MB

Table 3: Captured performance statistics for your solution.

6 Efficiency (5 Marks)

Marks for efficiency were assigned based on analysis of the submitted solutions to identify meaningful boundaries between types of solution. In particular, we set the following thresholds:

- **Speed (3 Marks):** Calculated based on Task Time. If task time was less than 30s then three marks were awarded. If less than 40s then 2 marks were awarded. If less than 50s then one mark was awarded. If the solution took longer than 50s then no marks were awarded.
- **Read Minimization (1 Mark):** If the solution reads less than 2,400MB then 1 mark was awarded.
- **Avoid Shuffling (1 Mark):** If the solution shuffles less than 200MB then 1 mark was awarded.

Component	Available	Benchmark	Mark
Speed	3	2	2
Read Minimization	1	1	1
Avoid Shuffling	1	1	1
Total			4

Table 4: Efficiency Marks Awarded

As a side-note, if you watched the video where I discuss my solution and you were wondering how you could have out-performed it to get full marks for speed, it is by using takeOrdered() rather than reducing to form the final ranking.

7 Generated Answer (5 Marks)

The final block of marks was awarded for generating one of the correct answer(s). As I summarized in the forum discussion, depending on how you do time filtering you can have a working solution but

generate slightly different answers. So I generated 11 valid answers, based on whether you include the recommendation day or not, filter to 365 or 366 days for the year or set the number of days for the Returns class to 4 or 5. I also included some cases where filtering to 251 entries was done rather than a time filter, but in this case you had to include assets with 251 days or less elements to rank correctly.

Marks were awarded based on automatic line-by-line matching against the valid solutions. Marks were awarded as follows:

- **Ranking (3 Marks):** Three marks were awarded for getting the correct ranking of assets, where this was determined by matching on the name of the asset in the output. If all 5 assets were correctly ranked 3 marks were awarded. If 4 assets were correctly ranked then 2 marks were awarded. If 3 assets were correctly ranked then 1 mark was awarded. Otherwise no marks were awarded.
- **Technical Indicators (1 Mark):** If the ranking was fully correct, then it was also possible to get a mark for including the technical indicator values (returns and volatility) correctly.
- **Asset Metadata (1 Mark):** If the ranking was fully correct, then it was also possible to get a mark for including the asset metadata correctly.

Component	Available	Mark
Output	5	5

Table 5: Mark awarded for solution generated output.

8 Summary

Group	Available	Mark
Functionality	10	10
Documentation + Efficiency	5	5
Performance	5	4
Output	5	5
Total	25	24

Table 6: Mark awarded for solution generated output.

A SPARK.DONE

StartTime:22 Mar 2025 03:05:47 GMT
 EndTime:22 Mar 2025 03:06:16 GMT
 Seconds: 28

Rank 1: TOP Ships Inc (TOPS)

- Industry: Marine Shipping
- Sector: Industrials
- Returns: 1.7777776673988128
- Volatility: 2.9349650430662058
- P/E Ratio: 0.84

Rank 2: Evolve Transition Infrastructure LP (SNMP)

- Industry: Oil & Gas Midstream
- Sector: Energy

- Returns: 1.0476191456085469
- Volatility: 3.0215562312976
- P/E Ratio: 0.34

Rank 3: Nautilus Inc (NLS)

- Industry: Leisure
- Sector: Consumer Cyclical
- Returns: 0.7999998684587193
- Volatility: 3.5138044029850057
- P/E Ratio: 1.08

Rank 4: Tilray Brands Inc (TLRY)

- Industry: Drug Manufacturers - Specialty & Generic
- Sector: Healthcare
- Returns: 0.7114428226577362
- Volatility: 3.7968434951482086
- P/E Ratio: 8.52

Rank 5: Battalion Oil Corp (BATL)

- Industry: Oil & Gas E&P
- Sector: Energy
- Returns: 0.6176470661027449
- Volatility: 2.5650400538676355
- P/E Ratio: 3.37

local-1742612748662

B SPARK.DATA (If Available)

```
[ {
  "id" : "driver",
  "hostPort" : "172.27.32.1:53533",
  "isActive" : true,
  "rddBlocks" : 0,
  "memoryUsed" : 134514,
  "diskUsed" : 0,
  "totalCores" : 4,
  "maxTasks" : 4,
  "activeTasks" : 0,
  "failedTasks" : 0,
  "completedTasks" : 28,
  "totalTasks" : 28,
  "totalDuration" : 33751,
  "totalGCTime" : 610,
  "totalInputBytes" : 2507255574,
  "totalShuffleRead" : 17943773,
  "totalShuffleWrite" : 17943773,
  "isBlacklisted" : false,
  "maxMemory" : 4924951756,
  "addTime" : "2025-03-22T03:05:48.693GMT",
  "executorLogs" : { },
  "memoryMetrics" : {
    "usedOnHeapStorageMemory" : 134514,
    "usedOffHeapStorageMemory" : 0,
    "totalOnHeapStorageMemory" : 4924951756,
    "totalOffHeapStorageMemory" : 0
  }
}
```

```
},
"blacklistedInStages" : [ ],
"peakMemoryMetrics" : {
  "JVMHeapMemory" : 884362512,
  "JVMOffHeapMemory" : 174675472,
  "OnHeapExecutionMemory" : 0,
  "OffHeapExecutionMemory" : 0,
  "OnHeapStorageMemory" : 246882,
  "OffHeapStorageMemory" : 0,
  "OnHeapUnifiedMemory" : 246882,
  "OffHeapUnifiedMemory" : 0,
  "DirectPoolMemory" : 73728,
  "MappedPoolMemory" : 0,
  "ProcessTreeJVMVMemory" : 0,
  "ProcessTreeJVMRSSMemory" : 0,
  "ProcessTreePythonVMemory" : 0,
  "ProcessTreePythonRSSMemory" : 0,
```

```
  "ProcessTreeOtherVMemory" : 0,
  "ProcessTreeOtherRSSMemory" : 0,
  "MinorGCCCount" : 230,
  "MinorGCTime" : 566,
  "MajorGCCCount" : 0,
  "MajorGCTime" : 0,
  "TotalGCTime" : 610,
  "ConcurrentGCCCount" : 18,
  "ConcurrentGCTime" : 44
},
"attributes" : { },
"resources" : { },
"resourceProfileId" : 0,
"isExcluded" : false,
"excludedInStages" : [ ]
} ]
```